yatra-agent: write-up

Sahil Verma · Agentic AI Engineer Intern take-home · github.com/sahilvermahr16-cmyk/yatra-agent

A budget trip planner for India. It takes a goal in plain words (English or Hinglish), writes a plan before acting, looks things up on live public APIs, recovers when a tool fails, keeps the trip inside the budget, and returns a Markdown and JSON report with a timed plan for every day. It is a LangGraph state machine with eight stages and gpt-4o-mini as the LLM.

Design decisions

1. **The LLM handles language and judgment; code handles every number.** The LLM is used in three places only: reading the goal, writing the plan, and choosing budget changes. Dates, rupees, distances, prices and timings are all computed in code, so every number in a plan can be tested (241 unit tests that need no network or API key).
2. **Plan first, then check the plan in code.** The planner writes an ordered tool plan with a reason for each step, and it is printed before any tool runs. Code checks it: one step per wish, the right order, and a weather step only inside the 16-day forecast range. A wrong plan gets one retry with the problems listed, then a default plan. Reading the goal works the same way: the LLM copies the user's own words for dates and budget, and code checks them against the goal.
3. **A failure becomes a note, not a crash.** Every HTTP call goes through one wrapper (timeout, retries, a trace event, failure injection), and every error is recorded with how the agent carried on. If the weather API is down, the trip is planned without weather. If a Wikipedia search fails, the time goes to free time rather than invented sights. If OpenStreetMap is busy, four servers are tried in turn and the days are still timed. An unclear goal stops with a reason.
4. **In the budget loop, the LLM weighs trade-offs and code does the arithmetic.** Code lists only the allowed changes, each with its exact saving. A must-visit place and a level the user named are never offered. The LLM chooses; code applies the changes, re-picks the hotel, meals and timeline from the saved map data, and checks the budget again (at most 3 rounds).
5. **Live data only, with the assumptions written down.** There is no free live API for Indian hotel and food prices or for visit lengths, so these are assumptions in `config.py`, and they are listed in every report.
6. **Measure before claiming.** The test sets were locked before running. Whole agent, dev set: 12/14 on the first run, 14/14 after three root-cause fixes. Holdout set, run once: 10/10 (95% CI 72–100%). Goal reading on its own: 50/57 across five sets. Reading the sample runs found one bug the checks could not see (a must-visit place matched to the wrong page), and it was fixed.

Limitations

- Prices, visit lengths and city travel speeds are assumptions, not live data.
- The free OpenStreetMap servers are often busy. When none answers, the plan names no hotels or restaurants (and says so).
- One base city per trip, with no origin city and no trains or flights. For a state, the base city is the most-read one, whatever the interests (Kerala + backwaters gives Thiruvananthapuram, not Alappuzha).
- Some budget phrasings are read unreliably ("about 15k" was right in 3 of 6 cases). A miss is treated as a maximum, which is the safe side.
- The evaluation sets are small (10–15 cases), so the intervals are wide. The checks test structure, not whether a chosen place is the best one.
- Known issue: when no cafe is found near the day's first place, the timeline shows the ride to that place before "breakfast near your stay".

What I would do differently with more time

- Use a places or hotel API (or a private Overpass server) for real prices, ratings and reliable availability.
- Support multi-city trips with an origin city and train or flight legs, and choose a state's base city by the user's interests.
- Build bigger, fresh holdout sets, add a human or LLM-judge check for "is this the right place", and compare other models.
- Fix the known issues: let the LLM list the people and count them in code, put breakfast before the first ride, and sharpen the "about" budget rule.
- Ask a follow-up question when the goal is unclear instead of stopping, and add a simple web interface.

I used an AI assistant to help while building this project.